Flash-SD-KDE: Accelerating SD-KDE with Tensor Cores

Elliot L. Epstein [1] **Rajat Vadiraj Dwaraknath** [1] **John Winnicki** [1] **Thanawat Sornwanee** [1]

Abstract

Score-debiased kernel density estimation (SD-KDE) achieves improved asymptotic convergence rates over classical KDE, but its use of an empirical score has made it significantly slower in practice. We show that by re-ordering the SD-KDE computation to expose matrix-multiplication structure, Tensor Cores can be used to accelerate the GPU implementation. On a 32k-sample 16-dimensional problem, our approach runs up to $40\times$ faster than a strong SD-KDE GPU baseline and $2{,}600\times$ faster than scikit-learn's KDE. On a larger 1M-sample 16-dimensional task evaluated on 131k queries, Flash-SD-KDE completes in 2.4 s on a single GPU, making score-debiased density estimation practical at previously infeasible scales. Code available at `https://github.com/Elliotepsteino/Flash-SD-KDE`.

1. Introduction

Score-debiased kernel density estimation (SD-KDE) (Epstein et al., 2025) improves the statistical efficiency of standard kernel density estimators, relative to classical kernel density estimation (KDE) (Rosenblatt, 1956; Parzen, 1962; Silverman, 1986): for sufficiently smooth densities, SD-KDE attains better bias and mean-squared error rates than vanilla KDE while retaining a simple, nonparametric form, while preserving nonnegativity. In particular, SD-KDE achieves an asymptotic mean integrated squared error (AMISE) of order $O(n^{-8/(d+8)})$, improving the convergence exponent relative to the $O(n^{-4/(d+4)})$ rate of classical KDE tuned with Silverman's rule of thumb (Silverman, 1986; Wand & Jones, 1995). These accuracy gains come with a significant computational drawback: the empirical score used for debiasing introduces an additional $O(n^2)$ pass over the data, so a naive implementation has roughly the same quadratic cost as KDE itself but with a larger constant factor.

Given samples $x_i \in \mathbb{R}^d$ and a bandwidth h, a Gaussian KDE is

$$\hat{p}(x) = \frac{1}{nh^d} \sum_{i=1}^{n} \varphi\left(\frac{x - x_i}{h}\right),$$

and SD-KDE forms debiased samples $x_i^{\mathrm{SD}} = x_i + \frac{h^2}{2}\,\hat{s}(x_i)$ where $\hat{s}$ is an estimate of the score. In this work we always use an empirical score computed from the KDE itself, for SD-KDE. Assuming a standard Gaussian kernel, φ, writing the KDE explicitly in terms of the samples, we estimate the score as

$$\hat{s}(x) = \frac{\nabla_x \hat{p}(x)}{\hat{p}(x)} = \frac{\sum_{i=1}^{n} \left[-(x - x_i)\,\varphi\left(\frac{x-x_i}{h}\right)\right]}{h^2 \sum_{i=1}^{n} \varphi\left(\frac{x-x_i}{h}\right)}.$$

2. Related Work

Kernel density estimation. Kernel density estimation (KDE) is a classical nonparametric approach to density estimation that dates back to the foundational work of Rosenblatt (1956) and Parzen (1962); modern treatments appear in standard references such as Silverman (1986) and Wand & Jones (1995). For smooth densities, KDE attains optimal nonparametric rates under appropriate bandwidth scaling, but in practice performance is highly sensitive to bandwidth selection. Rules-of-thumb and plug-in procedures are widely used for their simplicity, but they are typically tuned to unimodal, near-Gaussian targets and can misbehave for multimodal or heavy-tailed distributions (Silverman, 1986; Botev et al., 2010). This sensitivity is particularly pronounced in moderate and high dimensions, where the bandwidth controls both statistical bias and the computational cost of evaluating kernels at scale.

Bias reduction and adaptive smoothing. A large body of work improves KDE accuracy by reducing leading-order bias or adapting the effective smoothing scale across the sample space. One line of work constructs higher-order bias corrections by modifying kernels or combining estimators so that the dominant $O(h^2)$ bias term cancels, yielding $O(h^4)$ bias under additional smoothness assumptions (Fan & Hu, 1992; Jones & Signorini, 1997). Another class of methods uses variable bandwidths that shrink in regions of high density and expand in the tails, improving adaptivity without committing to a parametric form (Abramson, 1982; Silverman, 1986). Complementary to these ideas, diffusion-based estimators view density estimation through the lens

[1]Stanford University, Stanford, CA 94305, USA. Correspondence to: Elliot L. Epstein <epsteine@stanford.edu>.

Preprint. February 9, 2026.

of smoothing by a linear diffusion process, providing both adaptivity and a data-driven bandwidth selection mechanism (Botev et al., 2010). Score-debiased KDE (SD-KDE) (Epstein et al., 2025) fits into this landscape by using score information to correct the leading bias term while retaining a simple kernel estimator structure. Our Laplace-corrected formulation can be seen as an explicit higher-order correction that removes the need to form an empirical score at evaluation time, trading additional kernel evaluations for improved numerical and computational behavior.

Score estimation and score-based modeling. The score function $\nabla_x \log p(x)$ is a central object in statistics and machine learning. Score matching (Hyvärinen, 2005) provides a way to fit unnormalized models by matching scores rather than likelihoods, and denoising score matching connects score estimation to learning denoisers (Vincent, 2011). More recently, score-based generative models learn scores of noise-perturbed data distributions with neural networks and sample via Langevin or SDE-based dynamics (Song & Ermon, 2019; Song et al., 2021; Ho et al., 2020). These works typically target high-dimensional perceptual data and emphasize sampling or likelihood computation, whereas SD-KDE uses an explicit nonparametric density estimator and an analytically-defined score to improve pointwise density estimation. Related to our setting, Stein-based score estimators recover score information from samples via identities that avoid explicit density evaluation (Li & Turner, 2017). Our focus is complementary: we keep the statistical estimator fixed (SD-KDE and its Laplace variant) and address the practical bottleneck that arises when score-corrected estimators are deployed at large scale.

Fast evaluation of kernel sums and density models. The computational cost of KDE and related kernel methods is dominated by evaluating large Gaussian sums or kernel matrices. Algorithmic accelerations include multipole and series-expansion methods such as the Fast Gauss Transform (FGT) (Greengard & Strain, 1991) and improved variants tailored to Gaussian kernels (Yang et al., 2003). A different set of approaches uses randomized low-rank approximations (e.g., random Fourier features) to trade bias for computational efficiency (Rahimi & Recht, 2007). These methods reduce asymptotic complexity but can be sensitive to dimensionality, bandwidth, and target accuracy. In contrast, our work targets the exact SD-KDE computation and focuses on constant-factor reductions that make the estimator practical on modern accelerators.

GPU acceleration and tensor-core programming. A growing ecosystem of libraries and DSLs targets large kernel computations on GPUs. KeOps provides a memory-efficient reduction framework for kernel and distance matrices and is a strong baseline for Gaussian kernel reductions at scale (Charlier et al., 2021). General deep learning frameworks such as PyTorch (Paszke et al., 2019) make it easy to express quadratic kernel computations, but their performance depends critically on whether the computation can be lowered to high-throughput primitives. Domain-specific languages such as Triton (Tillet et al., 2019) enable writing custom GPU kernels while still benefiting from compiler support for tiling and scheduling. At the hardware level, modern GPUs provide specialized matrix-multiply units (Tensor Cores) and mixed-precision execution paths that can offer large throughput improvements when computations are expressed in GEMM-like form (Micikevicius et al., 2017; Williams et al., 2009). Recent "flash"-style algorithms demonstrate that reordering computations to avoid materializing large intermediate matrices and to match the GPU memory hierarchy can yield substantial speedups for quadratic primitives (Dao et al., 2022). Our contribution follows this direction for score-corrected density estimation: we reorder SD-KDE to expose matrix-multiplication structure and implement a streaming accumulation strategy so that Tensor Cores can be used effectively without incurring prohibitive memory traffic.

3. Hardware

All experiments run on a workstation equipped with an NVIDIA RTX A6000 (GA102). This GPU exposes 84 streaming multiprocessors (SMs); each SM contains 128 FP32 ALUs and 16 special function units (SFUs). Because the ratio of FP32 ALUs to SFUs is 128:16, we treat one exp (issued on an SFU) as costing the equivalent of $128/16 = 8$ FP32 flops in our models. The card delivers roughly 40 TFLOP/s of peak FP32 throughput and approximately 770 GB/s of GDDR6 bandwidth.

The host CPU is a dual-socket AMD EPYC 7763 system ("Milan") configured with 2×64 cores and two hardware threads per core (256 logical CPUs reported by `lscpu`). The processors boost up to 3.53 GHz, expose 512 MiB of shared L3 cache across the sockets, and provide modern ISA extensions (AVX/AVX2, FMA, BMI1/2, SHA, VAES, etc.).

4. Method

Our goal is to take n_{train} samples in d dimensions, form the empirical SD-KDE (score + shift), and evaluate the resulting density on n_{test} queries. Writing the computation in matrix form highlights the parallel structure. Let $x_i \in \mathbb{R}^d$ denote training points and $y_j \in \mathbb{R}^d$ denote query points, with bandwidth h and squared Euclidean distance

$$\|x_i - y_j\|^2 = \|x_i\|^2 + \|y_j\|^2 - 2\, x_i^\top y_j.$$

Stacking the training samples into $X \in \mathbb{R}^{n_{\text{train}} \times d}$, the empirical score computation is dominated by the train–train Gram

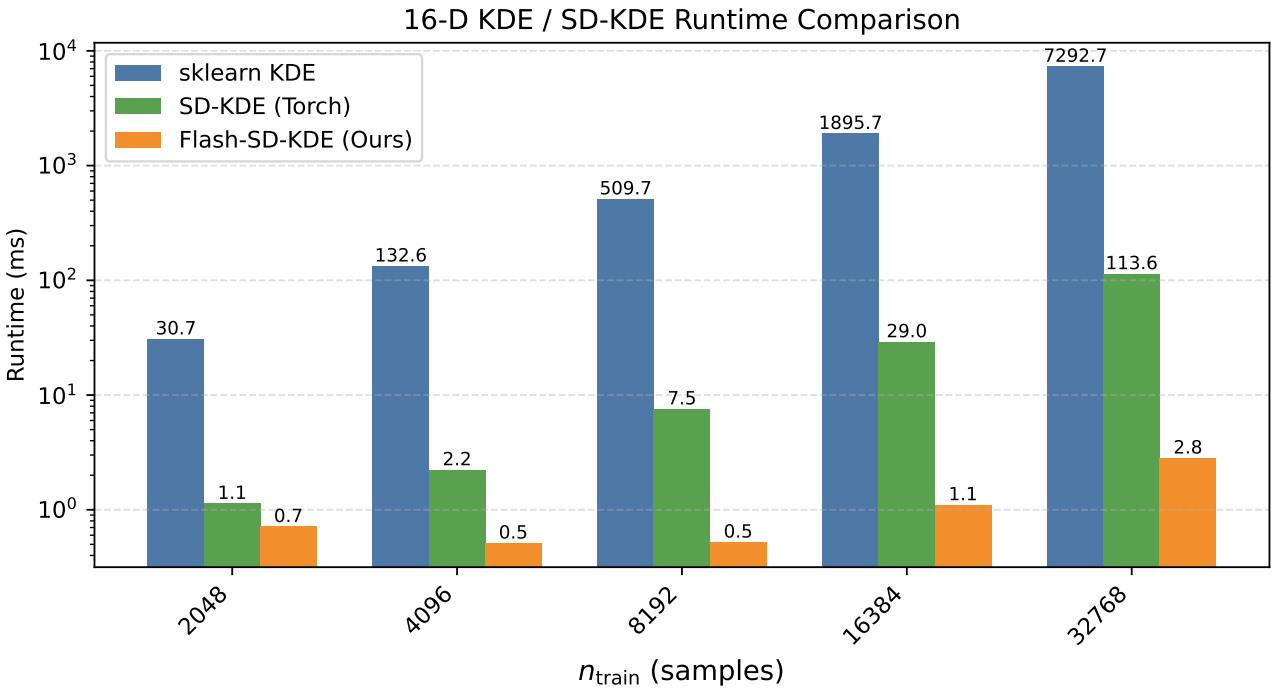

Figure 1. Runtime comparison for 16-D KDE/SD-KDE across n_{train} up to 32,768 ($n_{\text{test}} = n_{\text{train}}/8$).

matrix

$$G_{\text{score}} = XX^\top \in \mathbb{R}^{n_{\text{train}} \times n_{\text{train}}}.$$

Stacking queries into $Y \in \mathbb{R}^{n_{\text{test}} \times d}$ and denoting the debiased training samples by

$$X^{\text{SD}} = X + \tfrac{h^2}{2}\,\hat{s}(X),$$

the final SD-KDE evaluation uses the train–query Gram matrix

$$G_{\text{KDE}} = X^{\text{SD}} Y^\top \in \mathbb{R}^{n_{\text{train}} \times n_{\text{test}}}.$$

On modern NVIDIA GPUs this GEMM can be mapped to Tensor Cores (Micikevicius et al., 2017) and evaluated at 5–10× the throughput of standard FP32 SIMT arithmetic, particularly when we restrict to d that are multiples of 16 and use Triton's `tl.dot` interface (Tillet et al., 2019). The remaining operations—vector norms, broadcasted additions, and exponentials—are all $O(n_{\text{train}} n_{\text{test}})$ but quickly become a small fraction of the total FLOPs as d grows.

The numerator in the empirical score inherits a similar GEMM structure. Its numerator involves terms of the form

$$\sum_j -(x_i - x_j)\,\varphi_{ij}, \qquad \varphi_{ij} = \exp\left(-\frac{\|x_i - x_j\|^2}{2h^2}\right),$$

which naively suggests $O(n_{\text{train}} n_{\text{test}} d)$ additional elementwise arithmetic. Using the identity

$$\sum_j (x_i - x_j)\,\varphi_{ij} = x_i \sum_j \varphi_{ij} - \sum_j \varphi_{ij}\, x_j,$$

we can decompose the first term in the numerator into a elementwise sum, and the second term in the numerator into a second operation:

$$T = \Phi X.$$

Thus both the KDE evaluation and the SD-KDE score numerator reduce to Tensor-Core-accelerable matrix multiplies, plus $O(n^2)$ scalar work for the norms and exponentials. In this paper we focus on the $d = 16$ case, which aligns naturally with the Tensor Core tile sizes on the RTX A6000.

4.1. Arithmetic intensity in d dimensions

To understand when the high-dimensional SD-KDE becomes compute-bound, we estimate FLOPs, bytes moved, and arithmetic intensity for the d-dimensional case. Let $n_{\text{train}} = k$ and set $n_{\text{test}} = k/8$ as in the experiments.

Total FLOPs. The d-dimensional implementation consists of three main matrix-multiply stages:

1. Score Gram matrix $G_{\text{score}} = XX^\top$: $2dk^2$ FLOPs.
2. Score numerator $T = \Phi X$: $2dk^2$ FLOPs, plus $4k^2$ scalar FLOPs for norms and distance terms and $8k^2$ FLOPs for exponentials (counting each exp as 8 FLOPs due to the SFU/FP32 ratio).
3. Final KDE Gram matrix on debiased data: $2dk(k/8)$ FLOPs, plus $4k(k/8)$ scalar FLOPs and $8k(k/8)$ FLOPs for exponentials.

Aggregating these terms yields

$$\begin{aligned}\text{FLOPs}_d(k) &\approx 4dk^2 + 12k^2 + 2d\frac{k^2}{8} + 12\frac{k^2}{8} \\ &= \Big(4d + 12 + \tfrac{d}{4} + \tfrac{3}{2}\Big)k^2.\end{aligned}$$

Specifically for the $d = 16$ dimensional case, substituting $d = 16$ gives

$$\text{FLOPs}_{16}(k) \approx 81.5\,k^2,$$

which is on the order of 10^{11} FLOPs for $k = 32$k.

Bytes moved. To better capture implementation details, we count bytes per tile using the launch parameters that delivered the best runtime ($BLOCK\_M = 64$, $BLOCK\_N = 1024$). Each tile loads $64 \times d$ query values once ($4\,BLOCK\_Md$ bytes), streams $1024 \times d$ training values ($4\,BLOCK\_Nd$ bytes), and writes the partial PDF and weighted sums ($4\,BLOCK\_M$ and $4\,BLOCK\_Md$ bytes). All of this traffic goes to and from GDDR6, so for $d = 16$ a tile moves roughly

$$\begin{aligned}\text{Bytes}_{\text{tile}} &\approx 4\big(BLOCK\_Md + BLOCK\_Nd \\ &\quad + BLOCK\_M + BLOCK\_Md\big) \\ &= 4\big(2\,BLOCK\_Md + BLOCK\_Nd \\ &\quad + BLOCK\_M\big) \\ &\approx 7.4 \times 10^4 \text{ bytes.}\end{aligned}$$

The full problem processes $(k/BLOCK\_M) \times (k/BLOCK\_N)$ such tiles, so the total GDDR6 traffic is

$$\begin{aligned}\text{Bytes}_{16}(k) &\approx \text{Bytes}_{\text{tile}} \times \frac{k}{BLOCK\_M} \times \frac{k}{BLOCK\_N} \\ &\approx 1.13\,k^2 \text{ bytes.}\end{aligned}$$

Arithmetic intensity. Dividing FLOPs by bytes gives

$$\begin{aligned}I_d(k) &= \frac{\text{FLOPs}_d(k)}{\text{Bytes}_d(k)} \\ &\approx \frac{\big(4d + 12 + \frac{d}{4} + \frac{3}{2}\big)k^2}{4\big(\frac{9}{8}dk + \frac{k}{8}\big)} \\ &\sim C(d)\,k \quad \text{for large } k,\end{aligned}$$

with

$$C(d) \approx \frac{4d + 12 + d/4 + 3/2}{4 \cdot (9d/8)} = \frac{(17/4)\,d + 27/2}{9d/2}.$$

For $d = 16$ this simplifies to

$$I_{16}(k) \approx \frac{\text{FLOPs}_{16}(k)}{\text{Bytes}_{16}(k)} \approx \frac{81.5\,k^2}{1.13\,k^2} \approx 72 \text{ flops/byte.}$$

This tile-aware estimate more closely reflects the measured arithmetic intensity. Comparing against the A6000 specs (Tensor Core peak of 155 TFLOP/s versus $\approx$ 770 GB/s of memory bandwidth), the machine balance is roughly 200 flops/byte (Williams et al., 2009). Since our kernel sustains over 70 flops/byte, it lies well into the compute-bound regime on the RTX A6000. Nsight Compute reports an empirical arithmetic intensity of roughly 95 FLOPs/byte for the score kernel on the A6000, which is broadly in line with the 72 FLOPs/byte predicted by the tile-level model above. Minor differences stem from additional bookkeeping work (atomics, reductions) and from Nsight's finer-grained accounting of texture/cache traffic, but both figures agree that the kernel operates far above the bandwidth roofline. Because the kernel mixes Tensor-Core GEMMs with FP32 scalar work (norms, exponentials, atomics), the effective machine balance sits between the tensor-core roof ($\approx$ 200 flops/byte) and the FP32 roof ($\approx$ 50 flops/byte). The observed 70–95 flops/byte therefore straddle these two limits: the GEMM portion is partially bandwidth-limited relative to Tensor Cores, while the scalar portion is compute-limited relative to FP32 ALUs.

5. Laplace-corrected KDE

Score-debiased KDE offers better asymptotic error rates, but its empirical score step adds a full quadratic pass over the data. A natural approximation is to replace the explicit debiasing with a correction to the kernel itself, in the spirit of classical higher-order bias corrections for KDE[1] (Fan & Hu, 1992; Jones & Signorini, 1997). This yields a Laplace-corrected KDE that captures the same leading-order bias reduction of SD-KDE (Epstein et al., 2025) while avoiding the empirical score computation at the cost of admitting possibly negative values.

Let $K_h(x)$ denote the isotropic Gaussian kernel in d dimensions.

$$K_h(x) = \frac{1}{(2\pi)^{d/2}h^d}\exp\left(-\frac{\|x\|^2}{2h^2}\right),$$

Let $\hat{p}(x) = \frac{1}{n}\sum_i K_h(x - x_i)$ be the corresponding KDE. The Laplace-corrected kernel is defined by

$$K_h^{\text{LC}}(x) = K_h(x) - \frac{h^2}{2}\Delta K_h(x).$$

For the Gaussian kernel, the Laplacian has a closed form.

$$K_h^{\text{LC}}(x) = K_h(x)\left(1 + \frac{d}{2} - \frac{\|x\|^2}{2h^2}\right),$$

[1] Laplace kernel has 0 second moment, but non-zero fourth moment, so it is a fourth-order kernel.

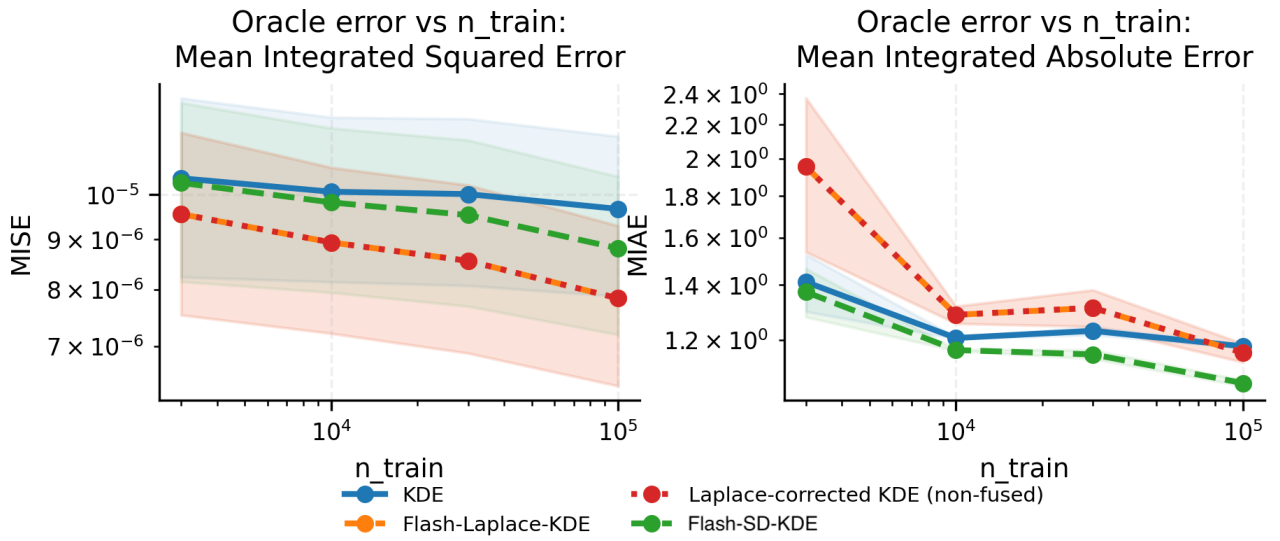

Figure 2. Oracle error on a 16D mixture-of-Gaussians benchmark. We report MISE and MIAE versus n_{train} for KDE, Flash-Laplace-KDE (fused Laplace correction), non-fused Laplace correction, and Flash-SD-KDE. The Laplace-corrected estimators can be slightly negative, so error is computed in a signed density manner.

The estimator becomes

$$\hat{p}_{\text{LC}}(x) = \frac{1}{n}\sum_{i=1}^{n} K_h^{\text{LC}}(x - x_i).$$

Connection to SD-KDE. SD-KDE (Epstein et al., 2025) forms debiased samples $x_i^{\text{SD}} = x_i + \frac{h^2}{2}\hat{s}(x_i)$ using an empirical score $\hat{s}$. Evaluating a KDE on the shifted samples and linearizing in h^2 yields a correction proportional to ΔK_h. When $\hat{s}$ is taken to be the KDE score, this linearized estimator coincides with the Laplace-corrected kernel above. Thus $\hat{p}_{\text{LC}}$ can be viewed as a first-order approximation to SD-KDE that preserves its leading bias reduction while avoiding an explicit score pass.

As an intuitive explanation of why both methods achieve the same leading bias reduction, we can first view vanilla KDE[2] as a heat semigroup (kernel) $P_t = e^{\frac{t}{2}\Delta}$ with $t = h^2$ applying to the empirical density function f_n (sampled with replacement) to be

$$\hat{f} = P_t f_n.$$

From the linearity of the heat semigroup, we then have that the expected estimate

$$\mathbb{E}\left[\hat{f}\right] = \mathbb{E}\left[P_t f_n\right] = P_t \mathbb{E}\left[f_n\right] = P_t f,$$

where f is the true probability density function.

The bias is then, under a smoothness assumption we assume throughout,

$$\mathbb{E}\left[\hat{f}\right] - f = (P_t - I)f = \frac{t}{2}\Delta f + O(t^2).$$

The Laplace kernel deals with this leading term by using an operator $\left(I - \frac{t}{2}\Delta\right) P_t$ instead, making its bias

$$\mathbb{E}\left[\hat{f}^{\text{LC}}\right] - f = \left[\left(I - \frac{t}{2}\Delta\right) P_t - I\right] f = O(t^2).$$

The SD-KDE algorithm approaches this debias task differently by applying a **non-linear**[3] transportation kernel $T_{\frac{t}{2}\nabla \log f}$ before applying a standard heat kernel P_t, making its bias

$$\begin{aligned}
&\mathbb{E}\left[\hat{f}^{\text{SD-KDE}}\right] - f \\
&= \left[P_t T_{\frac{t}{2}\nabla f} - I\right] f \\
&= \left[P_t\left(f - \frac{t}{2}\nabla \cdot (f \nabla \log f) + O(t^2)\right) - f\right] \\
&= \left[P_t\left(f - \frac{t}{2}\Delta f + O(t^2)\right) - f\right] \\
&= \left[P_t\left(I - \frac{t}{2}\Delta\right) - I\right] f + O(t^2) \\
&= O(t^2).
\end{aligned}$$

[2]Throughout, we assume that the kernel is Gaussian for simplicity.

[3]Even when the transportation kernel has a non-linear transportation. The kernel itself is linear.

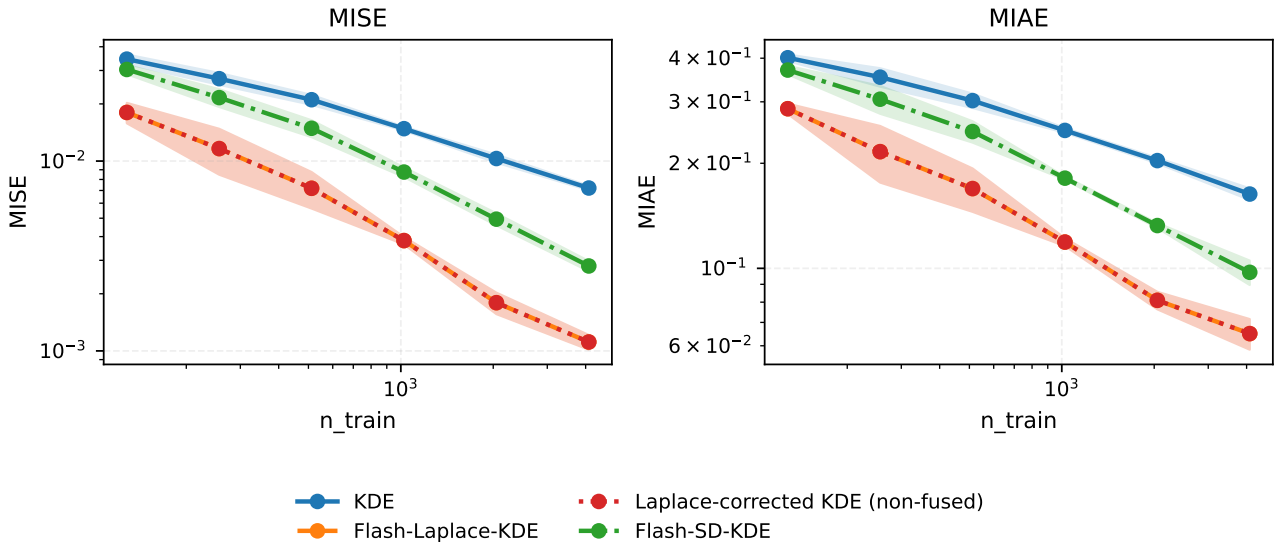

Figure 3. Oracle error on a 1D mixture-of-Gaussians benchmark. We report MISE and MIAE versus n_{train} for KDE, Flash-Laplace-KDE (fused Laplace correction), non-fused Laplace correction, and Flash-SD-KDE. The Laplace-corrected estimators can be slightly negative, so error is computed on the signed density.

Recall that $t = h^2$, so we have that Laplace KDE and SD-KDE have an $O(h^4)$ bias, while vanilla KDE has an $O(h^2)$ bias.

Note that the analysis above assumes that the SD-KDE has access to the oracle, so the translation kernel is linear. In practice, the score is empirical, making the translation kernel

$$T_{\frac{t}{2}\nabla \log(P_{t'} f_n)}$$

where $t' = \frac{h^2}{2}$ is the bandwidth for the kernel used for score estimation. Thus, the empirical SD-KDE is

$$\hat{f}^{\text{Emp SD-KDE}} = T_{\frac{t}{2}\nabla \log(P_{t'} f_n)} f_n,$$

which is no longer linear in f_n.

Why we consider this correction. The Laplace-corrected estimator is appealing when the full SD-KDE score is too expensive or when a fast bias-reduction surrogate is sufficient. It requires only the same pairwise distances and exponentials as standard KDE, plus a simple affine factor in the kernel value. The correction can be negative for large $\|x - x_i\|$, so $\hat{p}_{\text{LC}}$ is not guaranteed to be nonnegative, which we account for in our experiments.

Kernel fusion opportunity. Computing K_h^{LC} reuses the same scaled distances used to compute K_h. A fused kernel can compute $\phi = \exp(-\frac{1}{2}\text{ scaled})$ and apply the Laplace factor inside the same pass. A non-fused implementation must either recompute distances or materialize intermediates in a second kernel. This increases memory traffic and kernel launches. We therefore treat the fused Laplace-corrected kernel as a distinct fast path and refer to it as *Flash-Laplace-KDE*.

6. Results

We now summarize the empirical behavior of the 16-D implementation on the RTX A6000. For each n_{train} in $\{2\text{k}, 4\text{k}, 8\text{k}, 16\text{k}, 32\text{k}\}$ we set $n_{\text{test}} = n_{\text{train}}/8$, draw data from a simple 16-D Gaussian mixture, and run three baselines: scikit-learn KDE (Pedregosa et al., 2011), Torch SD-KDE (GEMM-based, implemented in PyTorch (Paszke et al., 2019)), and Flash-SD-KDE (Tensor-Core GEMMs for both KDE and score, implemented in Triton (Tillet et al., 2019)). Figure 1 shows that the Flash-SD-KDE rapidly outpaces both baselines as n grows.

We also measure utilization by combining the flop model above with the measured runtimes. As shown in Figure 5, the Flash-SD-KDE GPU utilization is high into the multi-digit range once n_{train} exceeds 8k, despite a lot of operations such as exponential computations that can't utilize tensor cores. This confirms that the 16-D Tensor-Core formulation is firmly compute-bound and that additional tuning effort should focus on kernel fusion and occupancy rather than memory traffic.

To compare against a specialized KDE implementation, we also benchmarked PyKeOps LazyTensor operators at $n_{\text{train}} = 32\text{k}$, $n_{\text{test}} = 4\text{k}$. PyKeOps currently represents the state of the art for large-scale kernel reductions, so we report

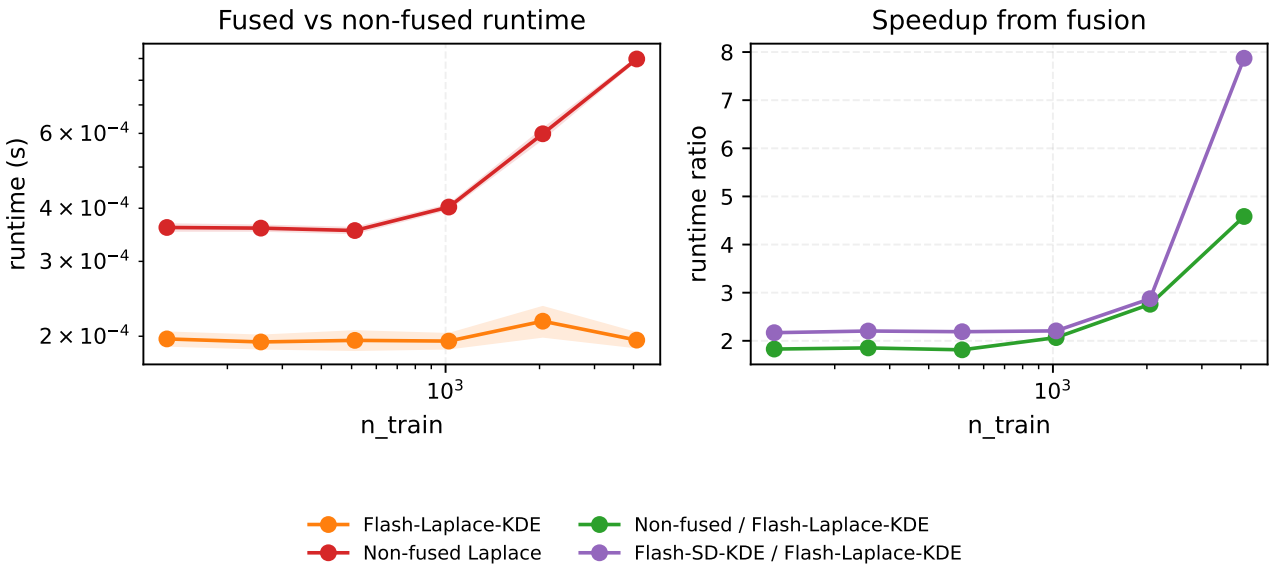

Figure 4. Runtime and speedup for Laplace correction in 1D. The left panel shows total runtime for the fused Flash-Laplace-KDE kernel and a non-fused implementation. The right panel reports speedup ratios, including Flash-SD-KDE relative to Flash-Laplace-KDE for context.

both its Gaussian KDE runtime and its SD-KDE runtime as a reference point. Table 1 shows that Flash-SD-KDE runs $1.4\times$ faster than the PyKeOps 16-D KDE baseline and $9.8\times$ faster than the PyKeOps 16-D SD-KDE implementation.

Method	Runtime (ms)	Rel. to Flash-SD-KDE
16-D Flash-SD-KDE	2.23	$1\times$
PyKeOps 16-D KDE	3.06	$1.4\times$
PyKeOps 16-D SD-KDE	21.73	$9.77\times$

Table 1. Runtime comparison at $n_{\text{train}} = 32\text{k}$, $n_{\text{test}} = 4\text{k}$. PyKeOps rows report Gaussian KDE and SD-KDE (LazyTensor) runtimes, while Flash-SD-KDE executes the full SD-KDE pipeline.

6.1. Oracle density benchmark for Laplace correction

Figure 2 reports oracle accuracy on the 16-D mixture benchmark as a function of n_{train}, using Mean Integrated Squared Error (MISE) on the left and Mean Integrated Absolute Error (MIAE) on the right. Across the sweep, Flash-SD-KDE substantially improves on vanilla KDE, indicating that the baseline KDE estimator is a poor fit in this regime. The Laplace-corrected variants achieve the lowest MISE throughout, and the fused Flash-Laplace-KDE curve overlaps the non-fused implementation, showing that fusion is an implementation optimization rather than a change in the estimator. In MIAE, Flash-SD-KDE attains the lowest error, while the Laplace-corrected estimators show a sharp drop between the smallest n_{train} and 10^4, followed by a small non-monotonic "kink" (a slight increase at intermediate n_{train}). The kink lies within the uncertainty bands and is consistent with finite-sample variability and the greater sensitivity of an absolute-error metric to the signed tail correction. Both metrics continue to decrease as n_{train} grows, but pushing this oracle evaluation much past $n_{\text{train}} \approx 10^6$ is difficult on a single RTX A6000 due to the cost of repeated large kernel-sum evaluations for MISE/MIAE. To probe this regime, Figure 8 extends the sweep using only KDE and Flash-Laplace-KDE, and we observe that Flash-Laplace-KDE continues the downward trend, achieving both MISE and MIAE well below KDE at $n_{\text{train}} > 10^6$. Because the Laplace-corrected kernel is not guaranteed to remain nonnegative, we compute MISE/MIAE on the signed estimator and separately log the integrated negative mass as a diagnostic.

We also evaluate Laplace-corrected KDE on a 1D oracle density. Figure 3 compares KDE, Flash-Laplace-KDE (fused Laplace correction), non-fused Laplace correction, and Flash-SD-KDE. The Laplace-corrected estimators can take small negative values for large $\|x - x_i\|$. We therefore compute errors on the signed density, and we report the negative-mass diagnostics separately in the benchmark logs.

Figure 4 shows the runtime advantage of fusion. The fused Flash-Laplace-KDE kernel consistently outperforms the non-fused Laplace correction across n_{train}. For context, the same plot also reports the Flash-SD-KDE to Flash-Laplace-KDE runtime ratio.

6.2. Performance optimizations

Nsight Systems traces show that roughly 95% of the SD-KDE runtime at $n_{\text{train}} = 32\text{k}$, $n_{\text{test}} = 4\text{k}$ is spent computing

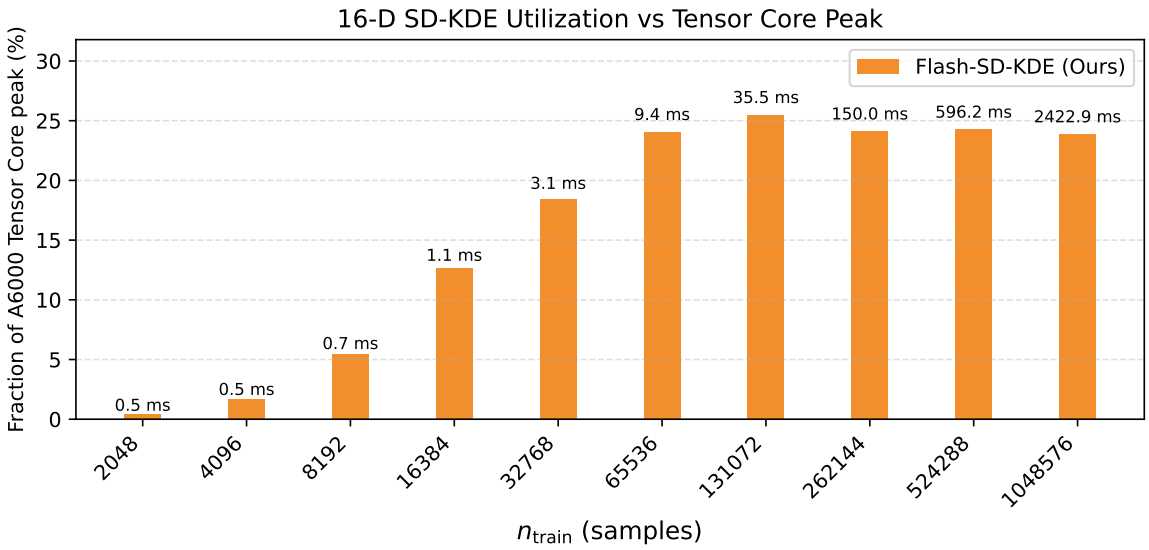

Figure 5. Utilization (percentage of RTX A6000 Tensor Core peak) for the 16-D SD-KDE pipeline, computed via the flop model from Section 4; bars are annotated with the observed runtimes (ms).

the empirical score. We therefore concentrated optimization effort on the score kernel, sweeping the launch parameters to raise utilization. Specifically we varied:

- $BLOCK\_M \in \{32, 64, 128, 256\}$,
- $BLOCK\_N \in \{32, 64, 128, 256, 512, 1024\}$,
- $num\_warps \in \{1, 2, 4, 8\}$,
- $num\_stages \in \{1, 2, 4\}$,

and selected the combination that minimized runtime. For this workload we found that $BLOCK\_M = 64$, $BLOCK\_N = 1024$, $num\_warps = 2$, and $num\_stages = 2$ gave the best overall performance, increasing measured FLOP utilization by more than $2\times$ relative to smaller-tile settings.

Beyond the launch-parameter sweep, two design choices account for most of the speedup over a standard tiling implementation:

1. **Tensor-Core tiling:** all high-dimensional dot products are processed as 16×16 tiles with Tensor Core acceleration, allowing the GEMM portion of the score and KDE kernels to run near the TF32 roofline.
2. **Streaming accumulation:** we never materialize full $n_{\text{train}} \times n_{\text{train}}$ (or query) matrices; instead we stream tiles through registers and rely on atomic reductions to keep global-memory traffic linear in n.

Together these optimizations push the kernels close to the hardware limit. Nsight Compute's "Speed of Light" report shows $\sim 68\%$ SM throughput and roughly 90% L1 throughput for the score kernel at $n_{\text{train}} = 32$k and $n_{\text{test}} = 4$k, which is consistent with the tensor-core utilization trends in Figure 5: we cannot reach the nominal Tensor Core peak because the kernel still executes substantial non-Tensor-Core work (norms, exponentials, atomics). At these utilization levels we stopped further low-level tuning, since additional optimizations would likely yield only modest incremental gains.

Acknowledgements

Large language models (LLMs) were used to assist both the implementation and the preparation of this paper. LLMs helped with kernel prototyping, refactoring, and drafting and editing text.

7. Conclusion

Score-debiased kernel density estimation (SD-KDE) offers improved asymptotic accuracy over classical KDE, but its reliance on an empirical score has historically made it substantially slower in practice. This paper shows that the dominant cost of empirical SD-KDE is not inherently prohibitive: by re-ordering the computation to expose matrix-multiplication structure, both KDE evaluation and the score numerator can be expressed as Tensor Core–accelerable GEMMs plus lightweight elementwise operations, while avoiding materializing full pairwise interaction matrices via streaming accumulation. In the 16-D setting, this hardware-aligned formulation yields large end-to-end gains—up to $40\times$ faster than a strong Torch SD-KDE baseline and $2{,}600\times$ faster

than scikit-learn KDE—and scales to previously infeasible regimes, completing SD-KDE on $\sim$1M training points and $\sim$131k queries in 2.4 s on a single GPU.

Overall, our results suggest that practical nonparametric estimators can often be unlocked by hardware-aware reformulations that maximize GPU utilization. Future directions include extending the approach beyond d multiples of 16, supporting richer bandwidth parameterizations and kernels, further reducing non-Tensor-Core overheads (e.g., exponentials and atomics), and developing nonnegativity-preserving approximations and multi-GPU/out-of-core variants.

Impact Statement

This paper presents work whose goal is to accelerate SD-KDE and KDE kernels on GPUs. We expect the primary impact to be enabling practitioners to apply these estimators to larger datasets; we do not foresee unique societal consequences outside the usual considerations for density estimation applications.

References

Abramson, I. S. On bandwidth variation in kernel estimates—a square root law. *The Annals of Statistics*, 10(4):1217–1223, 1982. doi: 10.1214/aos/1176345986.

Botev, Z. I., Grotowski, J. F., and Kroese, D. P. Kernel density estimation via diffusion. *The Annals of Statistics*, 38(5):2916–2957, 2010. doi: 10.1214/10-AOS799.

Charlier, B., Feydy, J., Glaunès, J. A., Collin, F.-D., and Durif, G. Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021. URL http://jmlr.org/papers/v22/20-275.html.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

Epstein, E. L., Dwaraknath, R. V., Sornwanee, T., Winnicki, J., and Liu, J. W. SD-KDE: Score-debiased kernel density estimation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=fvho95EtPu.

Fan, J. and Hu, T.-C. Bias correction and higher order kernel functions. *Statistics & Probability Letters*, 13(3):235–243, 1992. doi: 10.1016/0167-7152(92)90053-8.

Greengard, L. and Strain, J. The fast gauss transform. *SIAM Journal on Scientific and Statistical Computing*, 12(1):79–94, 1991. doi: 10.1137/0912004.

Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *arXiv preprint arXiv:2006.11239*, 2020.

Hyvärinen, A. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

Jones, M. C. and Signorini, D. F. A comparison of higher-order bias kernel density estimators. *Journal of the American Statistical Association*, 92(439):1063–1073, 1997. doi: 10.1080/01621459.1997.10474062.

Li, Y. and Turner, R. E. Gradient estimators for implicit models. *arXiv preprint arXiv:1705.07107*, 2017.

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

Parzen, E. On estimation of a probability density function and mode. *The Annals of Mathematical Statistics*, 33(3):1065–1076, 1962. doi: 10.1214/aoms/1177704472.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Rahimi, A. and Recht, B. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, 2007.

Rosenblatt, M. Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, 27(3):832–837, 1956. doi: 10.1214/aoms/1177728190.

Silverman, B. W. *Density Estimation for Statistics and Data Analysis*. Chapman and Hall, 1986. doi: 10.1201/9781315140919.

Song, Y. and Ermon, S. Generative modeling by estimating gradients of the data distribution. *arXiv preprint arXiv:1907.05600*, 2019.

Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2021. ICLR 2021.

Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. doi: 10.1145/3315508.3329973.

Vincent, P. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011. doi: 10.1162/NECO_a_00142.

Wand, M. P. and Jones, M. C. *Kernel Smoothing*. Chapman and Hall, 1995. doi: 10.1201/b14876.

Williams, S., Waterman, A., and Patterson, D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Yang, C., Duraiswami, R., Gumerov, N. A., and Davis, L. Improved fast gauss transform and efficient kernel density estimation. In *Proceedings of the Ninth IEEE International Conference on Computer Vision*, pp. 664–671, 2003. doi: 10.1109/ICCV.2003.1238383.

A. Flash-SD-KDE in low dimensions

For completeness we briefly summarize the 1-D SD-KDE arithmetic intensity and empirical behavior. With $n_{\text{train}} = k$ and $n_{\text{test}} = k/8$, there are two steps:

1. **Score + shift:** For each training point we compute $\hat{s}(x_i)$ from all k points and then form $x_i^{\text{SD}} = x_i + \frac{h^2}{2}\hat{s}(x_i)$. This uses $O(k^2)$ pairwise kernel interactions. With the RTX A6000 hardware ratio (128 FP32 ALUs, 16 SFUs per SM) we budget an $\exp$ as 8 flop-equivalents. The score accumulation requires one $\exp$ and roughly eight additional arithmetic ops (subtraction, scaling, accumulation), yielding $c_1 \approx 16$ flops per $(\text{train}, \text{train})$ pair.
2. **KDE on debiased samples:** We then evaluate a standard Gaussian KDE at $k/8$ query points using the k debiased samples, which uses $O(k^2/8)$ interactions. Each pair requires one $\exp$ (8 flops) plus about six other operations (difference, square, scaling, accumulation), so we take $c_2 \approx 14$ flops per $(\text{train}, \text{test})$ pair.

The total work is therefore approximated by

$$\text{FLOPs}(k) \;\approx\; c_1 k^2 + c_2\, k\,(k/8) \;\approx\; 16k^2 + 14\frac{k^2}{8} \;=\; 17.75\, k^2.$$

For $k = 32$k this is on the order of 2×10^{10} flops. When we fix $n_{\text{test}} = k/8$ we move approximately $5k$ bytes (counting one read of each train and test point and one write of each output), so the arithmetic intensity scales as

$$I(k) = \frac{\text{FLOPs}(k)}{\text{Bytes}(k)} \;\approx\; \frac{17.75\, k^2}{5\, k} \approx 3.55\, k \quad \text{flops/byte},$$

placing realistic problem sizes firmly in the compute-bound regime.

Empirically we sweep k over powers of two from 1024 to 64k, with $n_{\text{test}} = k/8$, and average over three seeds. For each configuration we record scikit-learn Gaussian KDE time, and SD-KDE Torch time, and Flash-SD-KDE time. Across the entire range, the Flash-SD-KDE implementation is consistently faster than scikit-learn, with speedups growing with k; at the largest sizes, Flash-SD-KDE achieves around 2 orders-of-magnitude speedup relative to the sklearn baseline, as shown in Figure 6.

As shown in Figure 7, we also compare the utilization of Flash-SD-KDE with SD-KDE (Torch) over a range of 1-D problem sizes, using powers of two for n_{train} up to $2^{16} \approx 64$ thousand (and $n_{\text{test}} = n_{\text{train}}/8$).

B. Oracle density benchmark for Laplace correction beyond 10^6

The results are shown below.

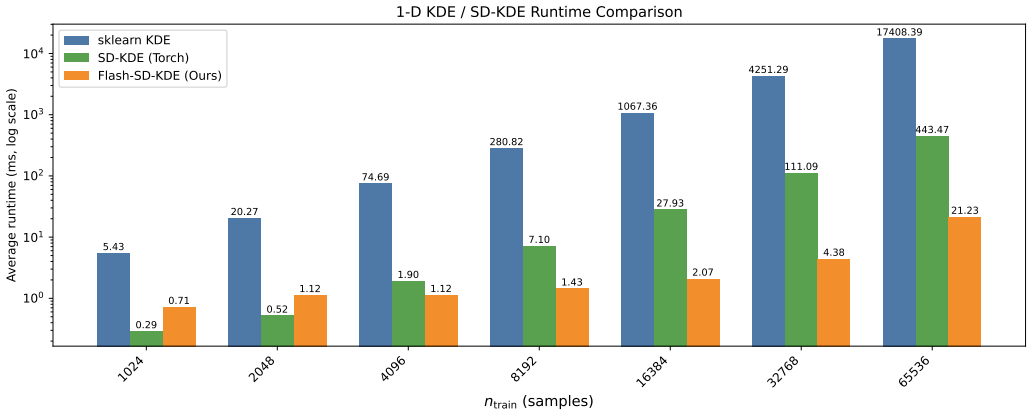

Figure 6. Average runtime (log-scale y-axis) of scikit-learn KDE, SD-KDE (Torch), and Flash-SD-KDE across n_{train} in 1-D; annotations show observed runtimes.

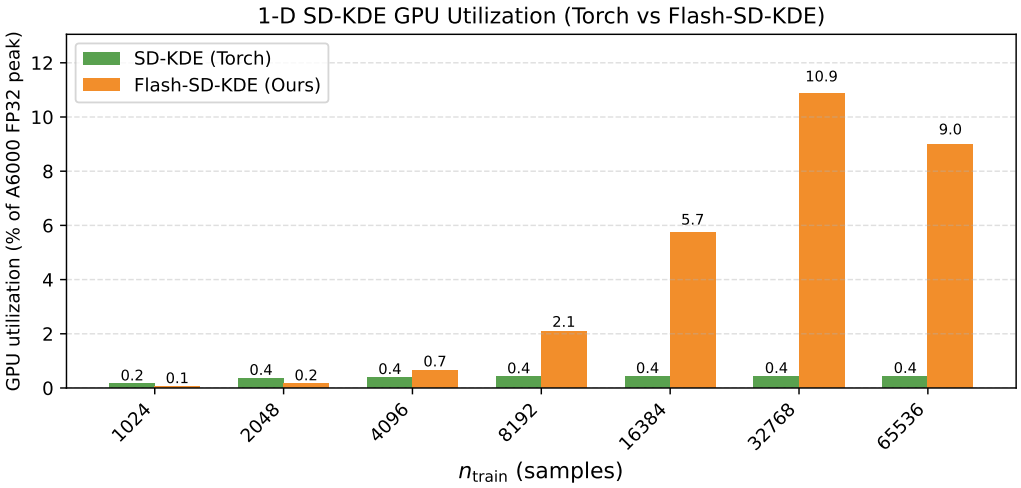

Figure 7. Utilization of the 1-D Flash-SD-KDE kernel and SD-KDE (Torch) for large n_{train} (powers of two up to 2^{16}). Labels show the observed runtime for each data size; error bars are omitted because a single seed is used per point.

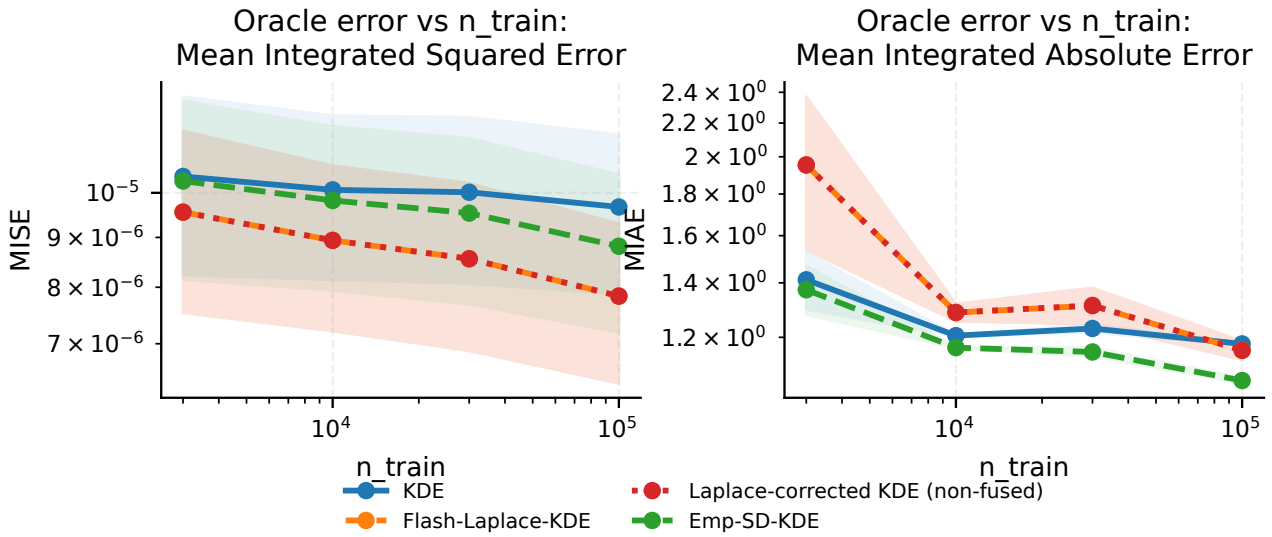

Figure 8. Oracle error on a 16D mixture-of-Gaussians benchmark. We report MISE and MIAE versus n_{train} for KDE and Flash-Laplace-KDE (fused Laplace correction).